



UNIVERSITY OF MINNESOTA

Driven to Discover®

Game Loop and Real-Time Simulation

CSCI 5980: Game Engine Architecture

Evan Suma Rosenberg | CSCI 5980 | Spring 2026

This course content is offered under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International license.



Today

- The rendering loop
- The game loop and a simple example
- Game loop architectural styles
- Abstract timelines (real, game, local, global)
- Measuring and dealing with time
- Multiprocessor game loops
- Job systems

Real-Time Simulation

- Games are **real-time, dynamic, interactive** computer simulations
- Time plays an incredibly important role:
 - Real time, game time, animation timelines, CPU cycles
 - Every engine system may define and manipulate time differently
- We need a solid understanding of all the ways time is used in a game

The Rendering Loop

- In a **GUI** (Windows, Mac), most screen contents are static
 - Only small portions change at any given moment
 - Traditionally drawn via **rectangle invalidation** (redraw only changed areas)
- **Real-time 3D graphics** work differently
 - The entire screen changes continually as the camera moves
 - Illusion of motion produced by presenting still images in rapid succession

The Rendering Loop

Producing rapid succession of still images requires a **loop**.

```
while (!quit)
{
    // Update the camera transform based on interactive inputs or by following a predefined path.
    updateCamera();

    // Update positions, orientations and any other relevant visual state of any dynamic elements in the scene.
    updateSceneElements();

    // Render a still frame into an off-screen frame buffer known as the "back buffer".
    renderScene();

    // Swap the back buffer with the front buffer, making the most recently rendered image visible on-screen.
    // (Or, in windowed mode, copy (blit) the back buffer's contents to the front buffer.
    swapBuffers();
}
```

The Game Loop

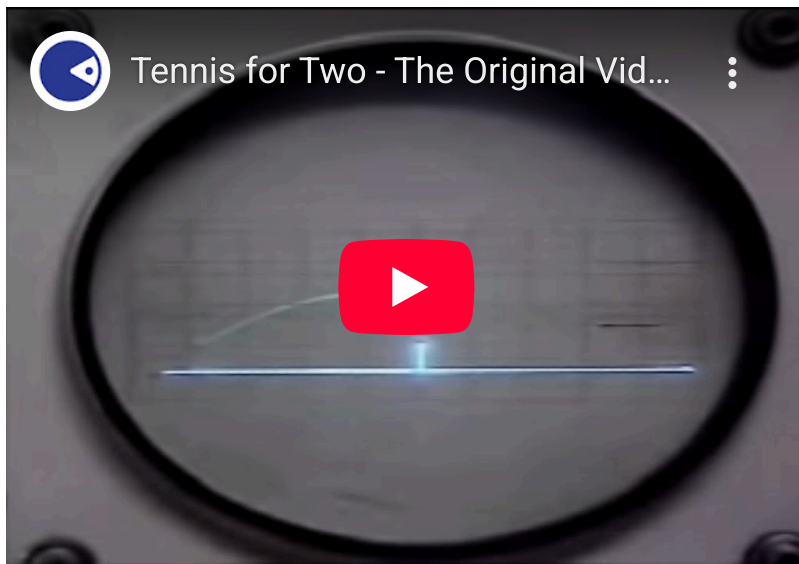
- A game is composed of **many interacting subsystems**:
 - Device I/O, rendering, animation, collision detection, physics, networking, audio
- Most subsystems require **periodic servicing** while the game runs
- Update rates vary by subsystem:
 - Animation: 30 or 60 Hz (syncd with rendering)
 - Physics: may need 120 Hz updates
 - AI: once or twice per second

The Game Loop

- Simplest approach: a **single loop** that updates everything
- This master loop is called the **game loop**
- It services every subsystem in the engine each iteration

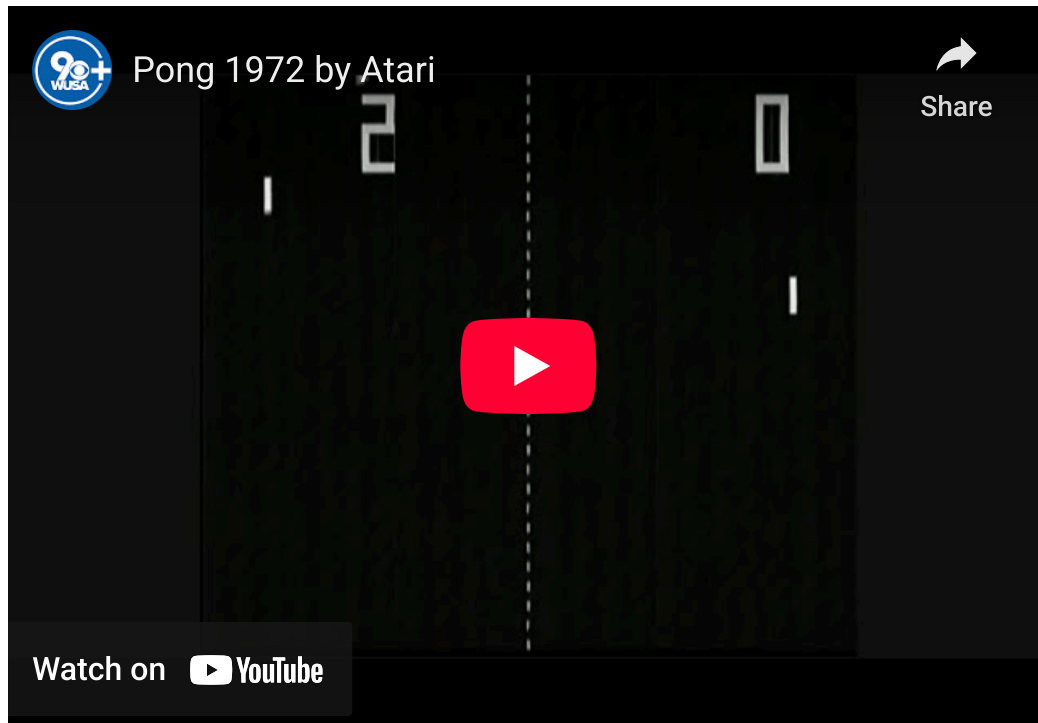
A Simple Example: Pong

- Pong genre started in 1958 as **Tennis for Two**
 - Analog computer game by William Higinbotham at Brookhaven National Lab
 - Displayed on an oscilloscope



A Simple Example: Pong

- Later incarnations include Magnavox Odyssey **Table Tennis**, Atari **Pong**



Pong Game Loop

```
void main() // Pong
{
    initGame();
    while (true) // game loop
    {
        readHumanInterfaceDevices();

        if (quitButtonPressed())
            break; // exit the game loop

        movePaddles();
        moveBall();
        collideAndBounceBall();
        handleBallImpact();

        renderPlayfield();
    }
}
```

```
void handleBallImpact() // Pong
{
    if (ballImpactedSide(LEFT_PLAYER))
    {
        incrementScore(RIGHT_PLAYER);
        resetBall();
    }
    else if (ballImpactedSide(RIGHT_PLAYER))
    {
        incrementScore(LEFT_PLAYER);
        resetBall();
    }
}
```

Game Loop Architectural Styles

- Game loops can be implemented in a number of different ways
- At their core, they usually boil down to one or more **simple loops** with various embellishments
- Common architectures:
 - Windows message pumps
 - Callback-driven frameworks
 - Event-based updating

Windows Message Pumps

- On Windows, games must also service **OS messages**
- Basic idea: service Windows messages whenever they arrive, run game loop only when no messages pending

```
while (true)
{
    // Service any and all pending Windows messages.
    MSG msg;
    while (PeekMessage(&msg, nullptr, 0, 0) > 0)
    {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }
    // No more Windows messages to process -- run one iteration of our "real" game loop.
    RunOneIterationOfGameLoop();
}
```

Windows Message Pumps

- **Side-effect:** Windows messages take precedence over the game loop
- Game temporarily **freezes** when you resize or drag the window
- A practical concern for PC game development

Callback-Driven Frameworks

- Some engines are structured as **frameworks** rather than **libraries**
- **Library**: suite of functions/classes called by the programmer
 - Maximum flexibility, but must understand proper usage
- **Framework**: partially constructed application
 - Game developer fills in missing functionality via callbacks
 - If closed-source, programmers have limited ability to change flow of control
 - If open-source, the programmer can modify the engine's flow of control

Callback-Driven Frameworks: OGRE Example

- **OGRE** rendering engine provides both library and framework interfaces
- Framework approach: derive from `Ogre::FrameListener`
 - Override `frameStarted()` and `frameEnded()`

```
while (true)
{
    for (each frameListener)
        frameListener.frameStarted();

    renderCurrentScene();

    for (each frameListener)
        frameListener.frameEnded();

    finalizeSceneAndSwapBuffers();
}
```

OGRE FrameListener Example

```
class GameFrameListener : public Ogre::FrameListener
{
public:
    virtual void frameStarted(const FrameEvent& event)
    {
        // Do things that must happen before the 3D scene is rendered
        // (i.e., service all game engine subsystems).
        pollJoypad(event);
        updatePlayerControls(event);
        updateDynamicsSimulation(event);
        resolveCollisions(event);
        updateCamera(event);
        // etc.
    }
    virtual void frameEnded(const FrameEvent& event)
    {
        // Do things that must happen after the 3D scene has been rendered.
        drawHud(event);
        // etc.
    }
};
```


Event-Based Updating

- An **event** is any interesting change in game state
 - Button press, explosion, enemy spotting the player, etc.
- Most engines have an **event system** for registering interest and responding
- Some engines leverage events for **periodic subsystem servicing**:
 - Event system must support **posting events into the future**
 - Handler performs work, then posts a new event 1/30 or 1/60 second ahead
 - Continues periodic servicing as long as required

Abstract Timelines

- Extremely useful to think in terms of **abstract timelines**
- A timeline is a continuous, one-dimensional axis
 - Origin ($t = 0$) can be placed at any arbitrary location
 - Implemented via a clock variable (integer or floating-point)
- Different timelines serve different purposes in the engine

Real Time

- Times measured via the CPU's **high-resolution timer register**
- Origin: the moment the CPU was last powered on or reset
- Measured in **CPU cycles** (or some multiple thereof)
- Can convert to seconds by multiplying by timer frequency

Game Time

- A timeline that is **technically independent** of real time
- Under normal circumstances, game time coincides with real time
- Powerful manipulation capabilities:
 - **Pause**: stop updating the game timeline
 - **Slow motion**: update game clock slower than real-time clock
 - **Scale and warp** one timeline relative to another

Game Time as a Debugging Tool

- Pausing game time **freezes the action**
 - Rendering engine and debug camera can continue running on a different clock
 - Developer can fly the camera around to inspect the world
- **Single-stepping**: advance game clock by one frame interval (e.g., $1/60$ s)
- Important: the game loop is still running when paused
 - Only the game clock has stopped
- Single-stepping the game clock $\neq$ setting a debugger breakpoint

Local and Global Timelines

- Animation or audio clips have a **local timeline**
 - Origin ($t = 0$) at the start of the clip
 - Measures time as originally authored/recorded
- Effects are visualized as a **mapping** between local and global timelines

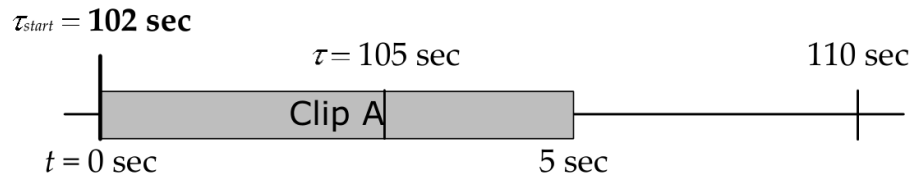


Figure 8.1. Playing an animation clip can be visualized as mapping its local timeline onto the global game timeline.

Timeline Mapping: Normal Playback

- To play a clip at **original speed**, map local timeline start ($t = 0$) to desired global start time ($\tau = \tau_{\text{start}}$)

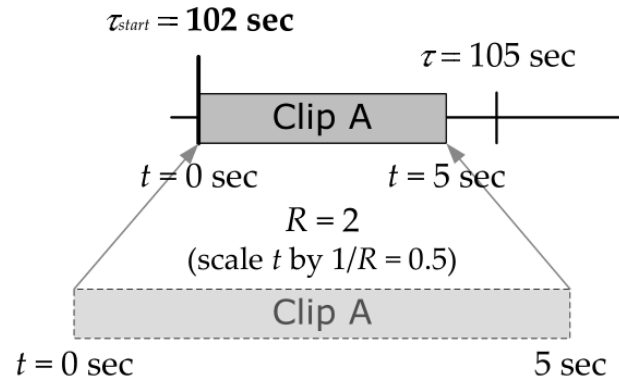


Figure 8.2. Animation playback speed can be controlled by simply scaling the local timeline prior to mapping it onto the global timeline.

Timeline Mapping: Scaled Playback

- Track a **time scale factor** (playback rate R) and global start time τ_{start}
 - $R = 2$: play at double speed (scale local timeline to half its size)
 - $R = 0.5$: play at half speed (scale local timeline to twice its size)
 - $R = -1$: play clip in **reverse**

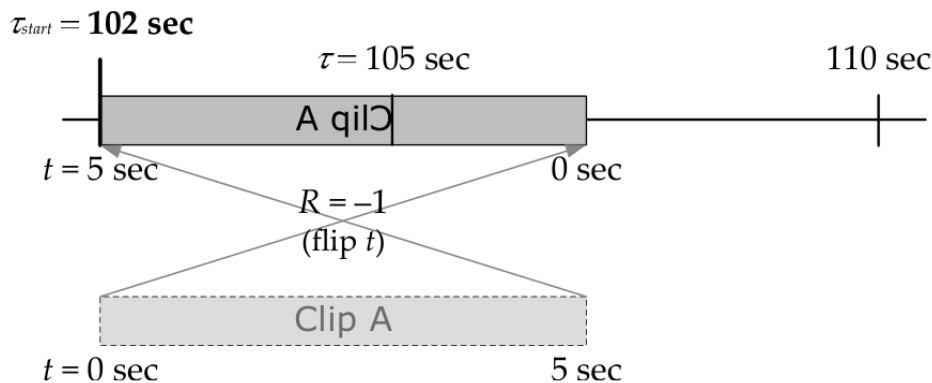


Figure 8.3. Playing an animation in reverse is like mapping the clip to the global timeline with a time scale of $R = -1$.

Measuring and Dealing with Time

- Several subtle and not-so-subtle distinctions between:
 - Different kinds of timelines
 - Different kinds of clocks
 - How they are implemented in real engines

Frame Rate and Time Deltas

- **Frame rate**: how rapidly still 3D frames are presented to the viewer
 - Measured in **FPS** (frames per second) or **Hertz** (Hz)
- Typical frame rates:
 - Film: 24 FPS
 - North America/Japan: 30 or 60 FPS (NTSC refresh rate)
 - Europe/rest of world: 50 FPS (PAL/SECAM refresh rate)
- Many displays are now capable of variable refresh rates (e.g., G-SYNC)
 - Allows for smoother gameplay and higher framerates (90-120 FPS)

Frame Time (Delta Time)

- **Frame time** (Δt): elapsed time between frames
 - Also called **time delta** or **delta time**
- At 30 FPS: $\Delta t = 1/30 \text{ s} \approx 33.3 \text{ ms}$
- At 60 FPS: $\Delta t = 1/60 \text{ s} \approx 16.6 \text{ ms}$
- To know actual elapsed time, we must **measure** it
- **Milliseconds** are a common unit in games
 - "Animation takes 4 ms" $\rightarrow$ about 12% of a 33.3 ms frame

From Frame Rate to Speed

- To move an object at constant speed v (meters/second):
 - $\Delta x = v \cdot \Delta t$ (change in position per frame)
 - $x_2 = x_1 + \Delta x$
- This is the **explicit Euler method** of numerical integration
- Works well for roughly constant speeds
- All integration techniques use Δt
 - *Perceived speeds* of objects depend on frame duration
- Central problem: **determine a suitable value for Δt**

Old-School CPU-Dependent Games

- Many early games made **no attempt to measure** real elapsed time
- Object speeds specified directly in distance **per frame** ($\Delta x = v \cdot \Delta t$) rather than per second (v)
- Result: perceived speed **entirely dependent on hardware frame rate**
 - Faster CPU → game runs in fast forward
- Some older PCs had a **"Turbo" button**
 - Pressed: full speed (CPU-dependent games too fast)
 - Not pressed: mimicked older processor speed

CPU-Dependent Example: Sierra Games



Note: this video is informative but contains a fair amount of profanity.

Updating Based on Elapsed Time

- To be **CPU-independent**, measure Δt each frame
 - Read high-resolution timer at start and end of frame
 - Subtract to get accurate Δt
- Pass Δt to all engine subsystems
 - via function parameters, global variable, or singleton
- Used by **most game engines**

Problem: Frame-Rate Spikes

- We use Δt from **frame k** as estimate for **frame k+1**
 - "Past performance is not a guarantee of future results"
- A **frame-rate spike** can cause problems
- Example: physics simulation stable at 33.3 ms (30 Hz)
 - One bad frame takes 57 ms
 - Next frame: physics steps twice to "cover" 57 ms
 - Two steps take ~twice as long → next frame is worse
 - Creates a **vicious cycle** of poor frame times

Using a Running Average

- Game loops tend to have **frame-to-frame coherency**
 - If camera points down a hallway this frame, likely still does next frame
- **Average** frame-time over a small number of frames
 - Use average as next frame's Δt estimate
- Trade-offs:
 - Longer averaging interval $\rightarrow$ less responsive but smoother
 - Shorter interval $\rightarrow$ more responsive but spikes have more impact

Governing the Frame Rate

- Alternative: **guarantee** every frame takes exactly the target duration (e.g., 33.3 ms)
- Approach:
 - Measure current frame duration
 - If less than target → **put main thread to sleep** until target time elapses
 - If more than target → "take your lumps" and wait one more whole frame time
- Called **frame-rate governing**

Governing the Frame Rate

- Only works when frame rate is **reasonably close** to target on average
- Still good practice to design systems for arbitrary frame durations
 - Use variable frame rate during development
 - Switch on governing when game consistently hits target
- Consistent frame rate benefits:
 - Physics integrators work better
 - Smoother visual experience
 - Enables reliable **record and playback**

Record and Playback

- Record every relevant event during gameplay with **accurate timestamps**
- Replay events with identical timing, initial conditions, and random seed
- Should produce gameplay **indistinguishable from original**
- If frame rate isn't consistent, events may not happen in the same order
 - Causes **drift** in replayed gameplay
 - AI characters may behave differently

Screen Tearing and V-Sync

- **Screen tearing**: visual anomaly when back buffer is swapped while screen is partially drawn
 - Portion shows new image, remainder shows old image
- To avoid tearing, wait for the **vertical blanking interval** (v-blank)
- V-blank: time during which the display resets between frames
 - Originally for CRT electron beam reset
 - Still exists in modern displays for compatibility

V-Sync Effects on Frame Rate

- **V-Sync** is another form of frame-rate governing
 - Clamps frame rate to a **multiple of screen refresh rate**
- On a 60 Hz NTSC monitor:
 - Frame rate quantized to multiples of $1/60$ s
 - Miss one v-blank $\rightarrow$ must wait for next: $2/60$ s (30 FPS)
 - Miss two $\rightarrow$ $3/60$ s (20 FPS)
- Don't assume 60 Hz
 - PAL/SECAM is 50 Hz

Measuring Real Time: High-Resolution Timer

- System time functions (e.g., `time()`) have **insufficient resolution**
 - `time()` returns seconds, which is far too coarse for frame timing
- All modern CPUs have a **high-resolution timer**
 - Hardware register counting CPU cycles (or multiple thereof)
 - On 3 GHz Pentium: resolution ≈ 0.333 nanoseconds

High-Resolution Timer APIs

- **Pentium:** `rdtsc` instruction (read time-stamp counter)
 - Win32 wraps this: `QueryPerformanceCounter()`, `QueryPerformanceFrequency()`
- **PowerPC** (Xbox 360, PS3): `mftb` instruction (move from time base register)
- Timer register is **64-bit** on most processors
 - At 3 GHz: wraps once every ~195 years
 - A 32-bit timer wraps after only ~1.4 seconds at 3 GHz

High-Resolution Clock Drift

- Even high-resolution timer measurements can be **inaccurate**
- On some **multicore processors**:
 - High-resolution timers are independent on each core
 - They can and do **drift apart**
- Comparing timer readings across cores may yield strange results
 - Even **negative time deltas**

Time Units and Clock Variables

- Two choices for any time measurement:
 1. **Time units**: seconds, milliseconds, machine cycles, or other?
 2. **Data type**: 64-bit integer, 32-bit integer, or 32-bit float?
- Answers depend on:
 - How much **precision** is needed
 - What **range of magnitudes** must be represented

64-Bit Integer Clocks

- Measured in **machine cycles**
- Extremely high precision (0.333 ns at 3 GHz)
- Extremely broad range (~195 years at 3 GHz)
- Most flexible time representation
- Trade-off: requires 64 bits of storage

32-Bit Integer Clocks

- Useful for measuring **short durations** with high precision (e.g., profiling)
- Store raw measurements in **64-bit** variables
- Only store the **delta** in 32 bits

```
// Grab a time snapshot.
U64 begin_ticks = readHiResTimer();

// This is the block of code whose performance we wish to measure.
doSomething();
doSomethingElse();
nowReallyDoSomething();

// Measure the duration.
U64 end_ticks = readHiResTimer();
U32 dt_ticks = static_cast<U32>(end_ticks - begin_ticks);

// Now use or cache the value of dt_ticks ...
```

32-Bit Floating-Point Clocks

- Store time deltas in **float** format, measured in **seconds**

```
// Start off assuming an ideal frame time (30 FPS).
F32 dt_seconds = 1.0f / 30.0f;

// Prime the pump by reading the current time.
U64 begin_ticks = readHiResTimer();

while (true) // main game loop
{
    runOneIterationOfGameLoop(dt_seconds);

    // Read the current time again, and calculate the delta.
    U64 end_ticks = readHiResTimer();

    // Check our units: seconds = ticks / (ticks/second)
    dt_seconds = (F32)(end_ticks - begin_ticks) / (F32)getHiResTimerFrequency();

    // Use end_ticks as the new begin_ticks for next frame.
    begin_ticks = end_ticks;
}
```

Limitations of Floating-Point Clocks

- IEEE 32-bit float: 23 bits shared between whole and fractional parts
- Small magnitudes → plenty of fractional precision
- Large magnitudes → whole part consumes more bits, fraction loses precision
- A floating-point clock tracking total elapsed time **will eventually become unusable**
- Best practice: only store **short time deltas** (a single frame or a few minutes)
 - Reset absolute clocks to zero periodically

Other Time Units

- Some engines use a custom game-defined unit, e.g., **1/300 second**
- Benefits:
 - Fine-grained enough for many purposes
 - Integer format (no floating-point needed)
 - 32-bit clock wraps only once every **165.7 days**
 - Even multiple of both NTSC (60 Hz) and PAL (50 Hz) refresh rates
- Limitations:
 - Not precise enough for subtle effects like animation time-scaling
 - Still need floating-point or machine cycles for those cases

Dealing with Breakpoints

- When a breakpoint is hit, the game loop stops
- Problem: the CPU timer keeps running
- After resuming, measured Δt can be seconds, minutes, or hours
- Passing a huge Δt to engine subsystems will cause unexpected behavior
 - Game lurches forward
 - Crazy physics calculations
 - Crashes

Dealing with Breakpoints

- Simple solution: if measured frame time exceeds an **upper limit**, assume a breakpoint was hit
- Set Δt artificially to target frame time (e.g., 1/30 s)
- Game becomes **frame-locked** for one frame to avoid the spike

Dealing with Breakpoints

```
F32 dt = 1.0f / 30.0f;
U64 begin_ticks = readHiResTimer();

while (true) // main game loop
{
    // Update and redraw code here

    // Read the current time again, and calculate an estimate of next frame's delta time.
    U64 end_ticks = readHiResTimer();
    dt = (F32)(end_ticks - begin_ticks) / (F32)getHiResTimerFrequency();

    // If dt is too large, we must have resumed from a breakpoint -- frame-lock to the target rate this frame.
    if (dt > 1.0f)
        dt = 1.0f/30.0f;

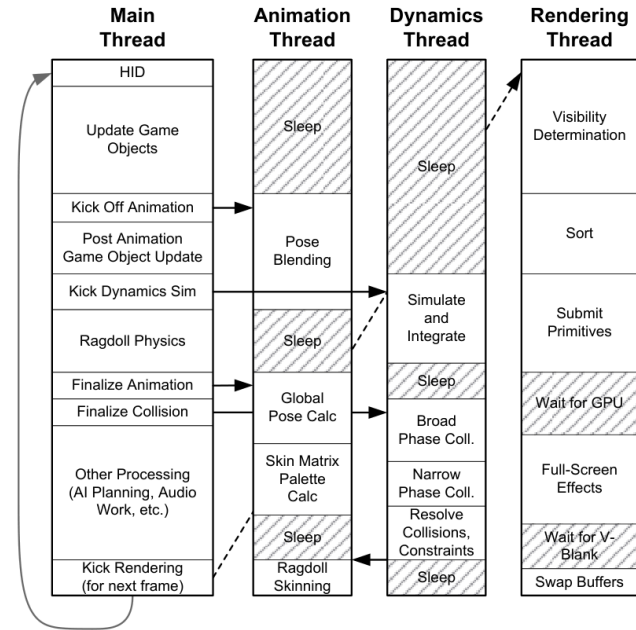
    // Use end_ticks as the new begin_ticks for next frame.
    begin_ticks = end_ticks;
}
```

Multiprocessor Game Loops

- Modern hardware provides **parallel computing** resources
- To take advantage, we must **decompose** game loop tasks into subtasks that execute in parallel
- Two categories of decomposition:
 - **Task parallelism**: different things done in parallel across cores
 - **Data parallelism**: same computation on many data elements in parallel

One Thread per Subsystem

- Simple approach: assign each engine subsystem to its own thread
- Master thread handles synchronization and main game logic
- This is a form of **task parallelism**



Problems with One Thread per Subsystem

- Number of subsystems likely **doesn't match** number of cores
 - Some threads must share a core via time-slicing
- Subsystems require **different processing amounts** each frame
 - Some cores idle while others are fully utilized
- **Dependencies between subsystems:**
 - Rendering can't start until animation/collision/physics complete
 - Can't run dependent subsystems in parallel
- Result: **not a practical design**

Scatter/Gather

- Many game loop tasks are **data-intensive**
 - Ray casts, animation blending, world-space matrix calculations
- **Divide and conquer** approach:
 - Instead of 9000 ray casts on one core
 - Divide into 6 batches of 1500, run each on a separate cor
- This is a form of **data parallelism**
- Also known as **scatter/gather** in distributed systems terminology

Scatter/Gather in the Game Loop

- Master game loop thread performs scatter/gather at various points during each frame
- Process:
 1. Divide N data items into m batches of $\sim N/m$ elements
 2. **Scatter**: Spawn m worker threads, each processing its batch
 3. Master thread can do other work while waiting
 4. **Gather**: wait until all workers complete (e.g., `pthread_join()`)
 5. Combine or finalize results as needed

Scatter/Gather in the Game Loop

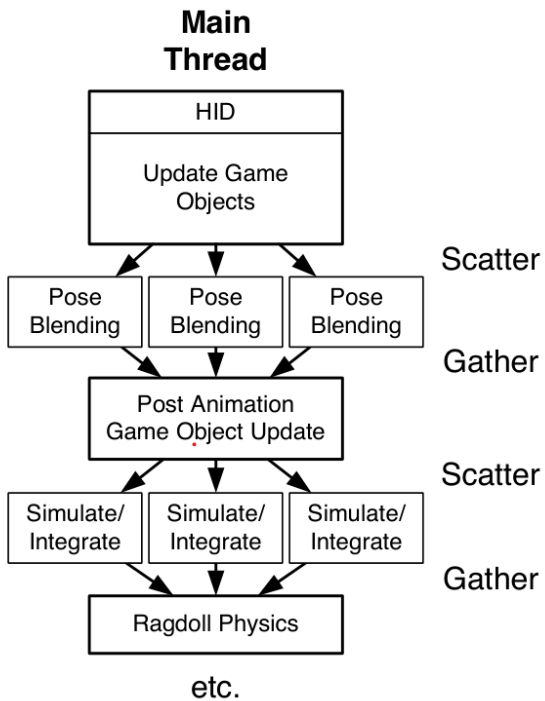


Figure 8.5. Scatter/gather used to parallelize selected CPU-intensive parts of the game loop.

Making Scatter/Gather More Efficient

- Problem: **spawning threads is expensive**
 - Kernel calls for creation and teardown
- Solution: use a **pool of pre-spawned threads**
 - Some OSes (Windows) provide thread pool APIs
 - Otherwise, implement with condition variables or semaphores
- Thread pool must be capable of performing **many different** scatter/gather operations per frame

Job Systems

- **General-purpose system** for executing arbitrary units of work across multiple cores
- Programmer subdivides each loop iteration into many independent **jobs**
- Job system maintains a **queue** and schedules jobs across available cores
- Like a lightweight OS kernel, but schedules **jobs** instead of threads

Benefits of Job Systems

- Jobs can be arbitrarily fine-grained
- Many jobs are independent of one another
- Helps maximize processor utilization
- Architecture scales naturally to any number of CPU cores

Typical Job System Interface

- **Kick a job**: equivalent of `pthread_create()`
- **Wait for jobs**: equivalent of `pthread_join()`
- **Self-terminate**: way for job to terminate early
- **Spin locks / mutexes**: for atomic concurrent operations
- **Sleep/wake**: via condition variables or similar

Job System Example

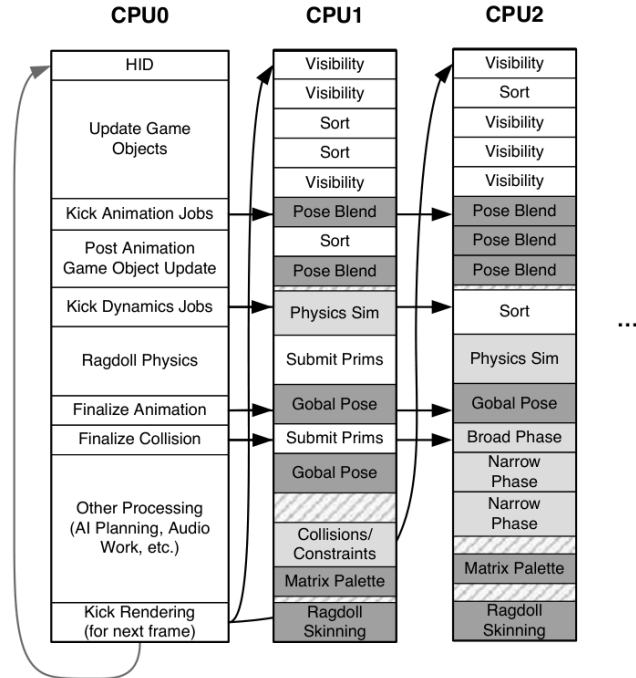


Figure 8.6. In a job model, work is broken down into fine-grained chunks that can be picked up by any available processor. This can help maximize processor utilization while providing the main game loop with improved flexibility.

Thread Pool-Based Job System

- Spawn **one thread per CPU core**, locked via affinity
- Each worker thread runs an **infinite loop**:
 1. Sleep waiting for a job request
 2. Wake up, call job's entry point function
 3. When job completes, return to top of loop
- Simple and effective for jobs that **run to completion**

Limitation: Jobs Can't Sleep

- In this system, each job **shares the call stack** of its worker thread
- A job cannot "go to sleep" waiting for results from another job
- Example: NPC AI job kicks off a ray cast, needs results before continuing
 - Can't yield and let other jobs run on the same worker thread
 - No way to save/restore the job's call stack

Jobs as Coroutines

- **Coroutines** can **yield** partway through execution
 - Continue from where they left off when control returns
- Implementation swaps call stacks of outgoing and incoming coroutines
- A coroutine-based job **can** go to sleep and let other jobs run

Jobs as Fibers

- **Fibers**: similar to coroutines
 - Context switches are always **cooperative**, never preemptive
 - Call stack saved when switching to another fiber
 - Fibers can migrate between threads
- Well-suited for job system implementation
- **Naughty Dog's** job system is based on fibers

Key Takeaways

- Games are **real-time simulations**
 - Time is central to everything
- The **game loop** is the master loop that services all engine subsystems
- Game loop architectural styles include:
 - Windows message pumps
 - Callback-driven frameworks
 - Event-based updating

Key Takeaways

- **Abstract timelines** provide powerful control over time
 - Real time, game time, local/global timelines
 - Pausing, slow motion, and reverse playback via timeline mapping
- Frame duration Δt drives object speeds via numerical integration
- Choosing how to measure and represent Δt is a central design decision:
 - Elapsed time, running average, or frame-rate governing
 - 64-bit integer, 32-bit integer, or 32-bit float clocks

Key Takeaways

- **Multiprocessor game loops** exploit parallel hardware
 - One thread per subsystem is simple but impractical
 - Scatter/gather provides data parallelism for intensive tasks
- **Job systems** are the modern approach
 - Fine-grained, general-purpose work scheduling across cores
 - Fiber-based implementations allow jobs to sleep and yield

No participation exercise today.

Assignment 0 is due on Thursday!



Questions?